

PandaSkill—Player Performance and Skill Rating in Esports: Application to League of Legends

Maxime De Bois , Flora Parmentier, Raphaël Puget, Matthew Tanti, and Jordan Peltier

Abstract—To take the esports scene to the next level, we introduce PandaSkill, a framework for assessing player performance and skill rating. Traditional rating systems, such as Elo and TrueSkill, often overlook individual contributions and face challenges in professional esports due to limited game data and fragmented competitive scenes. PandaSkill leverages machine learning to estimate in-game player performance from individual player statistics. Each in-game role is modeled independently, ensuring a fair comparison between them. Then, using these PScores, PandaSkill updates the player skill ratings using the Bayesian framework OpenSkill in a free-for-all setting. In this setting, skill ratings are updated solely based on performance scores rather than game outcomes, highlighting individual contributions. To address the challenge of isolated rating pools that hinder cross-regional comparisons, PandaSkill introduces a dual-rating system that combines players' regional ratings with a meta-rating representing each region's overall skill level. Applying PandaSkill to five years of professional *League of Legends* matches worldwide, we show that our method produces skill ratings that better predict game outcomes and align more closely with expert opinions compared to existing methods.

Index Terms—Esports, *League of Legends* (LoL), OpenSkill, player performance, skill rating, TrueSkill.

I. INTRODUCTION

RATING and ranking athletes has always been an integral part of competitive sports [1], [2]. Providing simple and quantifiable measures of player performance is beneficial for both professionals and fans. Such systems can be used by the industry for coaching and scouting future talents or to drive fan engagement by unveiling key game insights [3].

Recently, the increased availability of data has opened the way to more advanced, data-driven systems. Unlike traditional rating systems that rely on predefined formulas [4], data-driven approaches leverage large and fine-grained datasets with machine learning techniques to analyze player performance in a game [5], [6], [7], [8]. These methods can model more complex aspects of the game, such as the role of a given player or their contribution to the outcome of the game [6].

A similar need for performance evaluation exists in esports, with titles, such as *League of Legends* (LoL) or *Counter-Strike*, which require precise metrics to assess player skill [9]. Such

systems can benefit players through better matchmaking [10] and novel insights [11], [12], professional teams with quantified assessments of team performance [13], [14], and other actors in the ecosystem, such as betting operators [15], [16].

As video games, esports titles often have easier access to very detailed data compared to traditional sports, which often rely on tracking devices [17] or human annotations [18]. In fact, publishers often provide means to access the data, such as demo files for *Counter-Strike* or APIs for LoL and *Dota 2* [9], [12], [15], [16], [19].

On the other hand, the game state is generally far more complex [9]. For example, in LoL, a player chooses between more than 150 characters (called *Champions*) to play in a game. Each Champion has its own style of play and contributes differently to the team's victory. Players often play a specific role in a team, and even for a given role, it is common to see different playstyles develop [19]. Moreover, the game's balance is updated frequently, often adding new game mechanics. Something that was effective in one version might not be in the next.

Recent research has explored methods for evaluating player performance in games [11], [19]. Although analyzing a player's past performances can provide a reasonable estimate of their general skill level, it fails to capture the complete picture. A player's performance in a game is strongly influenced by the strength of their opponents. Facing weaker opponents may lead to exceptional performance, which might not accurately reflect their overall skill level. Algorithms, such as Elo, TrueSkill, or OpenSkill [20], [21], [22] are usually used to estimate the skill of a player, based on the outcomes of past games and the strength of the opponents. However, these algorithms do not take into account the individual contribution of a player to the outcome of the game. Moreover, in esports, the lower number of games played per player, compared to nonprofessional gaming, slows down the calibration of these algorithms. In addition, the scene is often fragmented, with some players rarely facing each other or even never at all. This raises the question of how to compare players from groups of players that have never played against each other. For instance, in LoL, apart from the two yearly international tournaments, players only face other players from the same region. This challenge, known as *isolated rating pools*, where player skill ratings evolve independently within isolated regions, has been acknowledged in chess literature and remains an open problem [23].

To address these challenges, we propose *PandaSkill*, a two-step framework to evaluate player performance and skill rating in esports. While the player performance represents the individual

Received 16 January 2025; revised 14 May 2025; accepted 12 June 2025.
Date of publication 19 June 2025; date of current version 17 December 2025.
Recommended by Associate Editor M. Seif El-Nasr. (Corresponding author: Maxime De Bois.)

The authors are with PandaScore, 75010 Paris, France (e-mail: maxime.debois@pandascore.co).

Digital Object Identifier 10.1109/TG.2025.3581070

contribution of the player to the outcome of a given game, their skill rating represent how good they are in general compared to other players. We can summarize our contributions as follows.

- 1) We propose an estimation of the player's performance in a game by using machine learning models that predict the outcome of the game from the player's statistics. By directly linking the performance score to the predicted probability, we make the methodology independent of the underlying model. Moreover, we ensure the fairness of the model by accounting for the different roles existing in the game.
- 2) The skill ratings of the players are computed from all the past performance scores (PScores). We use the Bayesian framework OpenSkill in a novel free-for-all (FFA) approach, where players are evaluated based on their performance relative to each other, independent of their teams. This way, the method focuses more on the players' individual performances.
- 3) We highlight the challenge of isolated rating pools in esports, hurting cross-regional comparisons. To alleviate the issue, we propose that the player skill rating be a combination of their contextual rating (within their region) and a meta rating (representing the skill level of the region). Improving cross-regional comparisons, this dual-rating system enables a more accurate evaluation of players on a global scale.
- 4) We apply the framework to five years of professional LoL matches from all the regions in the world. Compared to other methods in the literature, we demonstrate that our approach produces ratings that better predict the outcome of a game and are more in concordance with human experts.
- 5) The source code and data associated with this article can be found in the PandaSkill Github repository, alongside a web application to visualize the player performances and skill ratings. By making our tools publicly available at, we hope to encourage further research and practical applications in esports.

The rest of this article is organized as follows. In Section II, we review related work on player performance evaluation and rating systems in esports. We provide the reader with some background knowledge about LoL in Section III. Section IV details our proposed methodology. In Section V, we present the experimental setup, data used, and results of our experiments. We discuss the findings and limitations in Section VI. Finally, Section VII concludes this article.

II. RELATED WORK

In this section, we review existing data-driven player performance and skill rating evaluation systems. We focus mostly on esports or video game applications. However, since the field is relatively new, when relevant, we also discuss work from traditional sports.

A. Player Performance in a Game

Evaluating player performance in esports involves a variety of methodologies, ranging from basic metrics to the use of machine learning models.

The easiest way is to directly use one player statistic computed at the end of the game, such as the kill-death-assist ratio (KDA) in LoL [24], or kill-death ratio in *Counter-Strike* [25].

While easy to use, these metrics lack depth and do not tell the full story of what happened during the game. This led the community to craft more descriptive metrics, probably the most iconic of them in esports being the HLTV Rating for *Counter-Strike* [9], [15]. First introduced in 2010 and later improved with versions 2.0 and now 2.1, it combines player end-game statistics into a single value representing how much better the player performed in a given game compared to the average [25].

Inspired by traditional sports, Xu and Moka [26] proposed to use Plus/Minus values as an alternative to HLTV Rating 2.0, to better capture players' indirect contribution to team success. Plus/Minus value of a player is computed as the average point difference between his team and the opponent team.

Likewise, Demediuk et al. [19] proposed a framework to compute the current performance of a player during a game, which they called performance index (PI), for the esports game *Dota 2* (which is similar to LoL). The PI is computed as the weighted sum of the percentiles of the player's statistics. The weights are derived from random forest models trained to predict the game outcome from the player statistics. Multiple random forest models were trained, one per role and predefined playstyle archetype to account for the complexity of the game.

A similar methodology has been followed in *Soccer* with the PlayeRank framework [6]. They trained a single support vector classifier (SVC) model to predict the game outcome from team statistics (e.g., number of passes). The performance of a player is then computed as the share of the team statistic that can be attributed to the player. They also explored role-based and tournament-based variants of their framework, in which the weights are learned using only the corresponding subset of data (i.e., role-specific or tournament-specific samples).

Alternatively, researchers have explored the creation of more descriptive metrics, albeit not necessarily fully representative of the player performance. For instance, for *Counter-Strike*, Xenopoulos et al. [15] developed the win probability added (WPA) framework to value players' actions based on changes in their team's chances of winning. In LoL, Maymin proposed identifying *smart kills* and *worthless deaths* from the game events, as these are considered to have a strong impact on the outcome of the game [11]. Similarly, Jang et al. [27] explored how to convert player actions in LoL to quantitative scores using deep learning in order to better quantify individual contribution to team outcome.

B. Player Skill Rating

The simplest way to rate the skill of players and rank them is to average player metrics representative of their past performance. For instance, PlayeRank used the exponentially weighted moving average (EWMA) technique over the PScores of the player in past games, weighting recent performances higher [6]. On the other hand, Xenopoulos et al. [15] used averages of their proposed WPA metric and HLTV Rating 2.0 over a rolling window of one month.

However, averaging on recent player's performances does not take into account the strength of the opposing players in the calculation. Consequently, a high rating can be achieved by consistently competing against weaker opponents, resulting in an inflated ranking. This issue is still present in the tournament-based PlayeRank as it relies on learnt but fixed competition weights, ignoring actual opponent skill. In the context of chess, the Elo rating system has been widely adopted as a method to address this issue [20]. Elo adjusts a player's rating based on both the outcome of the match and the relative strength of the opponent, ensuring that victories against stronger opponents yield a greater rating increase, while wins against weaker opponents have a smaller impact.

The Glicko and later the Glicko-2 rating systems improve upon Elo by introducing a measure of uncertainty into a player's rating, making it one of the first Bayesian rating systems [28], [29]. The uncertainty represents the reliability of the rating. Each player starts with a high uncertainty value which slowly decreases game after game. Both Elo and Glicko ratings have been extensively used with applications in *Football* [1], [2], in casual LoL matchmaking [9], and in professional *Dota 2* teams rankings [16].

Nonetheless, both Elo and Glicko are designed for two-player games, which most esports titles are not. When applied to two-team games (which is very common), the individuality of the players is not considered. To address this problem in the context of casual matchmaking, Microsoft Research developed TrueSkill and TrueSkill2 to be used in first-person shooters (FPS), such as *Halo* or *Gears of War* [21], [30]. It differs from Elo and Glicko by updating the rating after each game instead of a given time period, and by working in multiteam/multiplayer contexts. One of the additions of TrueSkill2 is the use of additional metrics about the players, such as the number of kills scored or the tendency to quit the game.

A key downside of TrueSkill, however, is that it remains a proprietary framework. To provide a free and open-source alternative, OpenSkill was developed [22]. Thanks to the use of Bayesian approximation, OpenSkill is also faster, which is convenient in video games when there are a lot of matches to compute [31].

Another alternative to TrueSkill has been proposed by Delalleau et al. [10] to promote fun in casual matchmaking for the FPS *Ghost Recon Online*. They used a neural network to predict both balance and player enjoyment from the players' profiles (previous game statistics like number of kills or firing accuracy). Suznjevic et al. [32] investigated the addition of player's individual in-game accomplishments to the Elo ranking updates for the *Multiplayer Online Battle Arena* (MOBA) game *Heroes of Newerth*. Pradhan and Abdourazakou [16] explored a different approach, using a multicriteria decision-making system to rank *Dota 2* teams in a given season based on team metrics.

C. Position of Our Work

In this article, we propose a two-step end-to-end player skill rating evaluation. First, we compute the PScore of each player in a game from their end-game statistics. Then, we update the

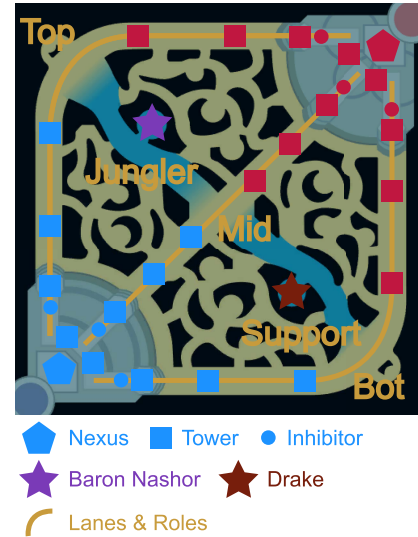


Fig. 1. Annotated minimap of LoL.

skill ratings of the players based on their respective performance in the game. This methodology is applied to five years of professional LoL.

When calculating PScores, our main goal is to base the evaluation on features that are most strongly tied to a team's chances of winning. Existing methods approach this by combining end-game statistics in a linear way. In contrast, we tie the PScore directly to the predicted probability of winning, estimated by a machine learning model that predicts the outcome of the game from the player's end-game statistics. This approach allows us to capture more complex relationships in the data, leading to a more accurate estimation of performance. Like the PI and PlayeRank approaches, we model each role independently to better capture its nuances [6], [19].

As for the estimation of the skill ratings, to the best of the authors knowledge, there exists no framework that simultaneously uses the player performance in-game whilst taking into account the opponents' pre-match skill level. Moreover, all rating systems are prone to the isolated rating pools challenge, where skill ratings of players evolve independently due to a low number of games between certain players [23]. As we highlight the impact of this issue on LoL professional ratings, we address it by combining a player's rating within their region with the rating of the region itself.

III. BACKGROUND ON LoL

A. Game Layout

LoL is a competitive MOBA game. In LoL, two teams of five players each compete on a map called *Summoner's Rift*, with the goal of destroying the enemy *Nexus*, located inside the enemy base and surrounded by *Towers* and *Inhibitors*. The two bases are connected by three paths, commonly called lanes: the top lane, the mid lane, and the bottom lane. Fig. 1 shows the simplified LoL map. At regular time intervals, *Minions* spawn in the two bases and push down the lanes, attacking nearby

enemy buildings. To destroy the enemy base, the players assist the Minions in their efforts. Between the lanes is the jungle, where neutral monsters can be defeated. The strongest ones, such as the *Drake* or the *Baron Nashor*, provide powerful bonuses to the team that defeats them.

B. Players

Each player embodies a character called a Champion. All Champions have a distinct set of abilities, strengths, and weaknesses. In a game, a player is also assigned a role. There are five different roles, one for each player: Top, Jungler, Mid, Bottom, and Support. The roles represent both the area of the map the player is supposed to play in during the early stages of the game (see Fig. 1) and the job of the player within the team. Here, is a basic general understanding of the roles as follows.

- 1) *Top*: Plays in the top lane; strong and independent; often provides tankiness and map control to the team.
- 2) *Jungle*: Plays between the lanes; moves around the map to apply pressure on the different lanes and help securing objectives.
- 3) *Mid*: Plays in the mid lane; has the most agency out of the laners in the early to mid game due to their central position in the map; often provides burst damage to the team.
- 4) *Bot*: Plays with the Support player in the early game in the bottom lane; provides sustained damage to the team; scales well into the late game by obtaining more gold.
- 5) *Support*: Plays with the Bot player in the early game in the bottom lane; does not need a lot of gold to perform; helps the team by providing vision and utility (e.g., healing, buffs, and control).

C. General Game Strategy

A typical game of LoL lasts between 20 and 50 min. During this time, each player tries to become stronger by leveling up and buying powerful items with the gold obtained by killing Minions, neutral monsters, enemy Champions, or buildings. Securing important neutral objectives, such as Drakes or the Rift Herald, also provides valuable advantages. When they are strong enough, they can attempt to break into the enemy base by sieging and fighting the enemy team together.

D. Esports Ecosystem

The highest competition tier of LoL is managed by the game publisher Riot Games. The world is divided into geographic regions, each having its own league and tournament format. Twice a year, on the occasions of the mid-season invitational (MSI) and the Worlds, the best teams from each region compete against each other. For years, the international scene has been dominated by Korea and China, followed by Europe and North America. As the esports scene matures, other regions (e.g., Asia-Pacific) are starting to emerge as serious contenders.

IV. PANDASKILL

PandaSkill computes the players' skill ratings in two steps. First, after each game, it computes the performance of the players

TABLE I
PLAYER SKILL RATINGS UPDATES BASED ON INDIVIDUAL PERFORMANCE (PScore) BY PANDASKILL FOR THE GAME 2 OF LCK SUMMER 2024 LOWER BRACKET FINAL, OPPOSING HLE TO T1 (VICTORY OF T1)

Player	Team	Role	PScore	Skill Rating
Viper	HLE	Bot	82.55	93.95 (+0.48)
Faker	T1	Mid	81.05	87.62 (+0.32)
Gumayusi	T1	Bot	73.96	90.42 (+0.28)
Peanut	HLE	Jungle	69.86	93.38 (+0.19)
Zeka	HLE	Mid	59.10	91.22 (+0.16)
Keria	T1	Support	58.62	91.81 (+0.05)
Delight	HLE	Support	52.94	92.50 (-0.11)
Oner	T1	Jungle	43.16	90.70 (-0.19)
Doran	HLE	Top	32.31	87.72 (-0.28)
Zeus	T1	Top	28.22	88.37 (-0.77)

in the game. Then, it uses the player performances to update the players' skill ratings. Table I shows a practical example of the framework applied to a game.

A. Measuring the PScore of Players in a Game

Measuring a player's performance in a game is challenging due to the absence of a known ground truth. Similarly to the PlayeRank and PI frameworks, we use the outcome of the game as a proxy for the player's performance [6], [19]. The idea is that if a player played well during a game, their performance should be reflected both in their end-game statistics and in the outcome of the game.

More specifically, we propose the following methodology to calculate the PScore of a player in a game.

- 1) For each role, we train a machine learning model that predicts the probability of a player winning the game based on their end-game statistics (e.g., number of kills).
- 2) The predicted probabilities are then transformed into percentiles. This transformation is learned for each model on the training data.

Compared to PI and PlayeRank, our approach has the following advantages.

- 1) Both PI and PlayeRank calculate the performance of the players by doing a linear combination of their end-game statistics, with weights that are derived from a model trained to predict the game outcome. By linking the PScores directly to the model's predicted probabilities, we account for the model's nonlinearity, enabling a more complex calculation of the player's PScore.
- 2) We ensure a fair cross-role comparison as all the scores are normalized on the same scale (from 0 to 100) by the percentile transformation, independent of the underlying model. This makes it possible to compare players with different roles, which is not the case for PlayeRank.
- 3) Our approach is model-agnostic as long as the model can output probabilities. Both PI and PlayeRank work with specific models from which feature importance can be easily computed (i.e., random forest for PI and linear SVC for PlayeRank).

Three qualities should be considered in order to choose which machine learning algorithm to use to compute the PScores as follows.

- 1) *Strong predicting capability*: A model without a good predicting capability will fail at capturing the nuances of the games.
- 2) *Calibrated probabilities*: The model should produce probabilities that closely reflect reality; it is particularly important as PScores are directly derived from the probabilities.
- 3) *Interpretability*: It should be possible to explain how the PScores are determined to ensure the system is transparent and trustworthy.

In this article, we used XGBoost models as they satisfy these qualities well.

B. Estimating the Skill Rating of the Players

1) *Openskill*: We base the evaluation of the skill rating of the players on the OpenSkill framework [22]. It has the advantage of being both open and working in multiteam or multiplayer settings, which is not the case for Elo, Glicko, or TrueSkill [20], [21], [28].

OpenSkill models the player's skill rating as a Gaussian distribution with the average skill of the player μ and an estimation of its uncertainty σ . Specifically, we represent the skill S_i of player i as

$$S_i \sim \mathcal{N}(\mu_i, \sigma_i^2). \quad (1)$$

The ratings of all players are initialized to the default values of $\mu_i = 25$ and $\sigma_i = 25/3$, as it is generally done in Bayesian rating systems [21], [22]. After a game, both μ_i and σ_i are updated based on the outcome of the player's team. The updates are derived from Bayesian inference, adjusting the player's skill distribution to reflect the new information provided by the game outcome. Formally, the update mechanism can be described by the following equation:

$$(\mu^{t+1}, \sigma^{t+1}) = \Omega(\mu^t, \sigma^t, \text{game outcome}) \quad (2)$$

where

- 1) (μ^t, σ^t) represent the ratings of all the players in the game before the game at time step t ,
- 2) $(\mu^{t+1}, \sigma^{t+1})$ denote their ratings after the game at time step $t + 1$, and
- 3) Ω is the Bayesian update made by the OpenSkill framework.

Since probabilistic estimates of skill ratings are often impractical to use directly (e.g., for creating rankings), they can be transformed into single-value estimates by taking the lower bound $\theta_i = \mu_i - 3 \cdot \sigma_i$. This provides a conservative estimate of the player's skill rating, ensuring a 99.7% likelihood that the player's true rating is higher.

2) *Free-for-All*: Natively, OpenSkill does not use the performance of the players to adjust the rating updates (which is also the case for Elo, Glicko, or TrueSkill). To incorporate the evaluation of the performance of the players in the game into the rating updates, we suggest framing the ratings updates as a FFA game. In this setting, all players compete against each other. After a game, they are ranked based on their individual PScore and their skill ratings are updated based on this ranking.

The update mechanism can then be written as follows:

$$(\mu^{t+1}, \sigma^{t+1}) = \Omega_{\text{FFA}}(\mu^t, \sigma^t, \text{PScore}) \quad (3)$$

where

- 1) **PScore** represents the PScores of the players in the game and
- 2) Ω_{FFA} is the Bayesian update made by the OpenSkill framework in the FFA setting.

Using the FFA setting has the following consequences.

- 1) The outcome of the game doesn't directly impact the update of the skill ratings. This means that a player can see their rating improve due to a good performance, even if they lost.
- 2) Rating updates depend on the relative performance the player has achieved compared to the other players in the game. To increase their rating, the player needs to perform better than the other players (regardless of their team).

3) *Contextual and Meta Ratings*: A significant challenge in global player ranking systems is the potential for ratings of players from different contexts (e.g., different regions or competition tiers) to evolve independently, creating isolated rating pools where direct comparisons become difficult. This is particularly problematic for some esports titles (e.g., LoL), where interregional play may be infrequent, limited to specific international tournaments.

To address this issue, we propose to model the skill rating of a player as the combination of two distinct ratings: a *contextual rating* and a *meta rating*. The contextual rating represents the player's skill within their context (e.g., region), while the meta rating represents the skill level of the context itself. This means that all players from the same context share the same meta rating.

We define the overall skill rating of a player as the sum of their contextual ($\mu_i^{\text{ctx}}, \sigma_i^{\text{ctx}}$) and meta ($\mu_i^{\text{meta}}, \sigma_i^{\text{meta}}$) ratings to capture both the skill of the player's within their context, and the context of the player

$$\mu_i = \mu_i^{\text{ctx}} + \mu_i^{\text{meta}} \quad (4)$$

$$\sigma_i^2 = \sigma_i^{\text{ctx}^2} + \sigma_i^{\text{meta}^2}. \quad (5)$$

After a game, using the OpenSkill framework, only one of the two ratings is updated depending on the context of the opponents in the match. If both teams share the same context, then the contextual rating of each player is updated. On the other hand, if the teams have different contexts (e.g., they are coming from different regions), we update the ratings of the teams' contexts. The following details how the updates are made in both cases.

- 1) *Updating contextual ratings*: If all opponents are from the same context, we update only the contextual ratings, because all the players in the game share the same meta rating, we do not need to use the meta ratings to update the contextual ones. As a consequence, updating the contextual ratings can be seen as standard rating update as described earlier. Formally, we can write

$$(\mu^{\text{ctx}, t+1}, \sigma^{\text{ctx}, t+1}) = \Omega_{\text{FFA}}(\mu^{\text{ctx}, t}, \sigma^{\text{ctx}, t}, \text{PScore}). \quad (6)$$

- 2) *Updating meta ratings*: If the opponents are from different contexts, we update only the meta ratings. Contrary to the contextual rating updates, here we use the contextual ratings as offsets to the meta ratings to take into consideration the individual strength of the players in the game. The offsetting technique has been used in TrueSkill2 as an efficient way to balance the skill ratings of players playing as a squad and solo players [30]. The meta ratings update can be written as follows:

$$(\mu^{\text{meta},t+1}, \sigma^{\text{meta},t+1}) = \Omega_{\text{FFA}} (\mu^{\text{meta},t} + \theta^{\text{ctx},t}, \sigma^{\text{meta},t}, \text{PScore}) \quad (7)$$

where $\theta^{\text{ctx},t}$ are the players lower bound contextual ratings in the game.

However, there are two issues with (7). First, all the resulting $\mu_i^{\text{meta},t+1}$ are shifted by $\theta_i^{\text{ctx},t}$. We need to reverse the offset to preserve the original scale of $\mu_i^{\text{meta},t}$. Moreover, $\mu^{\text{meta},t+1}$ contains multiple different meta rating updates for players sharing the same context. These variations arise from the variability in the players' contextual ratings and their individual performances in the game. In order to have the same updated meta rating for players that share the same context, we need to derive a single updated meta rating for each context in the game. We chose to average the updates for the same context as it ensures that the meta rating reflects the collective contributions and performances of all players associated with the context. The post-processing of the updates can be written as follows:

$$\mu_i^{\text{meta},t+1} = \frac{1}{n} \cdot \sum_{j=1}^n (\mu_j^{\text{meta},t+1} - \theta_j^{\text{ctx}}) \quad (8)$$

$$\sigma_i^{\text{meta},t+1} = \sqrt{\frac{1}{n} \cdot \sum_{j=1}^n (\sigma_j^{\text{meta},t+1})^2} \quad (9)$$

where j represents a player in the game, and n is the number of players in the game.

Note that, in both (6) and (7), while we used the FFA OpenSkill setting, we can also use the standard OpenSkill, which bases the updates on the game outcome, if we prefer.

Now that we know how to update the contextual and meta ratings, we need to handle the following edge case. When a player's context changes (e.g., the player goes to another team in another region), their contextual rating might not be representative to the player's skill level in this new context. When that happens, we suggest resetting σ_i^{ctx} to its default value of 25/3 to represent increased uncertainty in the contextual rating of the player in this new context.

C. Ranking and Player Comparison

PandaSkill can be used to rank and compare players. In particular, the probability of player i being better than player j can be estimated using the cumulative distribution function Φ

$$P(S_i > S_j) = \Phi \left(\frac{\mu_i - \mu_j}{\sqrt{\sigma_i^2 + \sigma_j^2}} \right). \quad (10)$$

TABLE II
DESCRIPTION OF THE DATASET FOR MAJOR COMPETITIONS

Competition	Region	Editions	Games	Players
Worlds	Global	5	592	353
MSI	Global	4	312	166
LCK	Korea	10	2,438	198
LPL	China	12	3,643	283
LEC	Europe	15	1,315	149
LCS	North America	13	1,371	182
PCS	Asia-Pacific	10	1,295	235
VCS	Vietnam	12	1,489	158
CBLOL	Brazil	10	1,153	196
LLA	Latin America	12	890	175
...				
Total		392	37,388	4,927

To rank all the players, instead of using the player comparison probability estimation, which would be too computationally heavy, we can use the lower bound $\theta_i = \mu_i - 3 \cdot \sigma_i$ as a single-value estimate. While this approach is computationally efficient and easy to interpret, it may result in lower θ_i values for players with fewer games (e.g., new professional players or substitutes) due to higher uncertainty (σ_i). However, this ensures that the rankings reflect not only performance but also the confidence in the player's skill rating.

V. EXPERIMENTS

In this section, we describe the experiments we conducted applying PandaSkill to the esports title LoL. The general goal is to assess the fairness and relevance of the PScores and skill ratings, both across roles and regions, because there is no known ground truth for neither of them, we must rely on proxy metrics and experiments.

A. Data

1) *Raw Data*: We used publicly available data from professional LoL games obtained through the Leaguepedia API [33]. The dataset comprises games from all the professional regions worldwide, spanning five years from 15 September 2019 to 15 September 2024. The end date coincides with the end of the last major regional tournaments before the start of Worlds 2024. Table II provides a detailed view of the composition of this dataset. We note the large discrepancy between the number of games in inter-region tournaments (Worlds and MSI) compared to regional ones.

For each game, we have the raw end-game statistics of the players (e.g., amount of gold, number of kills), events (e.g., player killing another player), game-related data (e.g., duration), and metadata (e.g., competition the game was in). Each player in a game has been assigned a region based on their participation in the latest highest tier regional tournaments.

2) *Feature Extraction*: From the raw data, we extracted features that both reflect metrics widely used by players and facilitate modeling. Table III describes all the features we used.

Most of the data points are normalized using reference values specific to each game. The two most common normalizations are the game-length normalization and total-kills normalization.

TABLE III
DESCRIPTION OF PLAYER STATISTICS USED AS INPUT FEATURES TO THE PSCORE MODELS

Feature	Description
KLA ratio	Ratio of enemy player kills compared to deaths. Computed as $(kills + assists)/(deaths + 1)$.
Gold per minute	Rate at which the player obtained gold in the game.
Experience per minute	Rate at which the player gained experience in the game.
Creep score per minute	Rate at which the player slays Minions or neutral monsters, procuring gold.
Wards placed per minute	Rate at which the player placed wards to provide vision for the team.
Damage dealt total kills ratio	Amount of damage dealt to players, normalized by the total number of kills in the game.
Damage dealt per gold total kills ratio	Player damage dealing efficiency, computed as <i>Damage dealt total kills ratio</i> further normalized by gold obtained.
Damage taken total kills ratio	Amount of damage taken from players, normalized by the total number of kills in the game.
Damage taken per gold total kills ratio	Player damage tanking efficiency, computed as <i>Damage taken total kills ratio</i> further normalized by gold obtained.
Largest multi kill	Maximum number of player kills in a short time window.
Largest killing spree total kills ratio	Maximum number of successive player kills without dying, normalized by the total number of kills in the game.
Worthless death ratio	Ratio of player deaths that did not benefit the team in any way.
Free kill ratio	Ratio of player kills that resulted in a worthless death for the enemy.
Objective contest winrate	Share of neutral objectives contested and won by the player.
Objective contest loserate	Share of neutral objectives contested but lost by the player.

Game-length normalization accounts for different game durations (e.g., having 10 kills in 15 min is not the same as in 50 min). Total-kills normalization accounts for different pacing in the game, as some games can have much more fighting than others.

Particular attention has been given to using features or data points that are strongly linked solely to the player's performance, because of that, we have excluded all team-related statistics (e.g., total number of kills by the team), and decided against using team-normalization techniques (e.g., share of number of kills within the team).

Some features described in Table III deserve more attention as follows.

- 1) *Kill-life-assist ratio (KLA)*: This formula has been preferred over the more traditional KDA ratio for numerical stability. Indeed, both formulas $KDA = (kills + assists)/deaths$ or $KDA = (kills + assists)/\max(deaths, 1)$ are either instable when $deaths = 0$ or cannot differentiate between 0 and 1 death.
- 2) *Worthless death ratio and Free kill ratio*: A player's death is considered worthless if, within a 1-min window, the player has not been involved in an enemy kill or their team has not secured an objective (e.g., a Drake or a Tower). The latter describes situations where the player's death gave the opportunity to their team to accomplish something else important on the map. Note that, while it shares the same core idea as the worthless death described by Maymin [11], we preferred a less complex implementation as it is not the focus of this article.
- 3) *Objective contest winrate and loserate*: These features are computed based on neutral monster kill events. An event is considered contested if players from both teams are present among the killer or assists of the event, with the contest win assigned to the team of the killer. The rates are calculated by normalizing the number of contest wins and losses by the total number of contestable objectives in the game. It is important to note that, due to the lack of event location data in the dataset, fights occurring near a neutral objective are not categorized as contests for that objective unless explicitly registered as part of the neutral monster kill event.

B. Computing and Evaluating the Players' Game Performance

First, we consider the computation of the player's PScores.

1) *Implementation Details*: As discussed in Section IV-A, we implemented our proposed approach using multiple XGBoost models, one per role. Specifically, they were trained with 2000 boosted rounds and a learning rate of 0.01.

To improve the interpretability of the model, we enforce monotonicity constraints for all the features. These constraints force each feature to impact the output probability in only one direction. For instance, an increase in the KLA should only increase the win probability, not decrease it. This prevents the model from finding hidden patterns in the combination of features that could lead to unexpected uses of certain features. In practice, of all the features we used, only the Worthless death ratio and Objective contest loserate are forced to decrease the output probability with higher values.

A five-fold cross-validation was used to compute the performance of the players and evaluate the models. Before being input to the models, all the features were standardized to zero mean and unit variance.

2) *Experimental Results*: Evaluating the PScores is difficult because of the lack of known ground truth. In these experiments, three aspects of the models are looked at: their predictive capability, the calibration of the predicted probabilities, and the interpretability of the outputs.

First, regarding the predictive capability, our XGBoost models achieve an average accuracy of 90.74% (Standard Deviation, $SD = 0.59$), comparable to role-based SVC models (90.99% accuracy, $SD = 0.47$) from the PlayeRank framework [6], and random forest models (91.31% accuracy, $SD = 0.57$) from the PI framework [19]. These high accuracy values are expected, as we are predicting game outcomes based on end-game statistics, which contain extensive information about in-game events. The slightly lower accuracy of our models can be attributed to the enforcement of monotonicity constraints. Without these constraints, our models achieve a higher accuracy of 91.79% ($SD = 0.56$). However, this improved performance comes at the cost of using unintended feature interactions, which undermine the explainability of the PScores (e.g., a higher creep score per minute paradoxically decreasing the predicted winning probability).

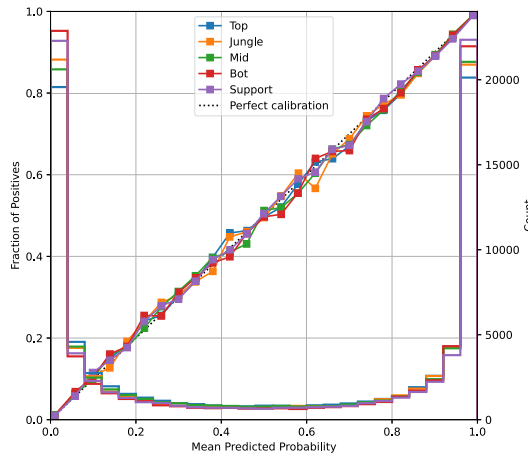


Fig. 2. Calibration plots with histograms of predicted probabilities for each role.

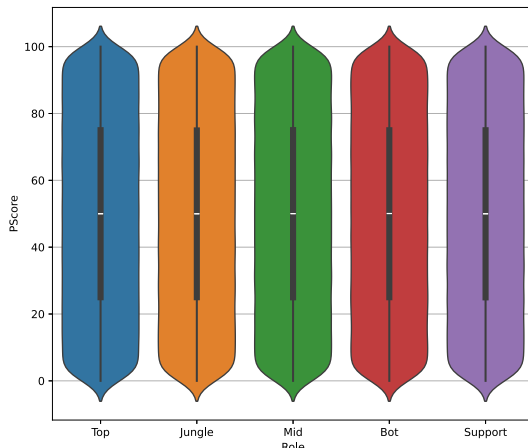


Fig. 3. Distribution of players' PScores for each role.

As mentioned in Section IV-A, the good calibration of the model is very important, as the player performance is directly related to the trustworthiness of the probabilities. We used the Expected Calibration Error (ECE) to measure quality of the estimated probabilities [34]. With a mean ECE (of 0.93% (SD = 0.03)), the model is fairly well calibrated. In comparison, the PlayeRank approach has an average ECE of 1.28% (SD = 0.11) and the PI approach an average ECE of 2.29% (SD = 0.07). Fig. 2 displays the calibration plots of our models for the different roles, alongside the histogram of the probability values. The U-shape of the probability distribution, combined with its variability across different roles, shows the need for the percentile transformation in the calculation of PScore to achieve uniformly distributed game performances across all roles. Fig. 3 shows the result of this transformation across roles. We can see that, thanks to the percentile mapping, all roles are treated equally.

Finally, we examine the importance of the different features used to compute the PScores. To quantify each feature's contribution to the predictions, we use SHapley Additive exPlanations (SHAP) values [35]. SHAP is a model-agnostic explainability

method that measures the average change in a model's predictions when a given feature is included or excluded. SHAP values can explain individual predictions, as shown in Fig. 7, or represent general behavior through their distribution over games and roles, as illustrated in Fig. 4. The KLA emerges as the most important feature across all roles. Other significant features include the Free kill ratio, Experience per minute, Gold per minute, Worthless death ratio, and Objective contest loserate. This highlights not only the importance of well-known metrics like Gold per minute but also the relevance of new features we introduced, such as the Free kill ratio. Besides, Fig. 4 shows the impact of the monotonicity constraints that have been enforced in the XGBoost models, as all the features are pushing the winning probabilities in the expected directions.

3) *Conclusion*: In summary, our approach implemented with XGBoost has a good predictive capability while being calibrated and maintaining a good interpretability. These properties validate PScore as a reliable and interpretable basis for the skill-rating experiments that follow.

C. Computing and Evaluating the Player Skill Ratings

1) *Experimental Models*: In this section, we are interested in how well our OpenSkill-based framework estimates the player's skill ratings. To measure the impact of the different modules we proposed, we consider the following model variations.

- 1) *OpenSkill*: The skill rating updates only depend on the team outcome of the game and not the player's in-game performance.
- 2) *FFA_OpenSkill*: The skill rating updates use the FFA setting described in Section IV-B2.
- 3) *Meta_OpenSkill*: The skill rating is defined as the combination of both contextual and meta ratings as described in Section IV-B3.
- 4) *Meta_FFA_OpenSkill*: The skill rating is both defined as the combination of both contextual and meta rating, and use the FFA setting in the update.
- 5) *Meta_FFA_TrueSkill*: Similar to the Meta_FFA_OpenSkill variation, but with TrueSkill instead of OpenSkill. We specifically used the non official implementation of Lee, as the official one is not available [22], [36].

In addition, as it is used in the PlayeRank framework [6], we compare these rating models to EWMA, which we implemented with a smoothing factor of 0.05 to put more emphasis on the player's performance history.

We apply these rating models to the PScore, PlayeRank, and PI performance models.

2) *Prematch Game Outcome Forecasting Experiment*: First, we evaluate the models by their ability to forecast the outcome of the game from the players' skill ratings before the game. The intuition is that more accurate skill ratings should be better predictors of the outcome of the game.

To account for the gradual changes in the rating distribution and potential meta shifts in the video game, we used a rolling window of one month. We trained a logistic regression model on one year of data and tested it on the subsequent month. Then,

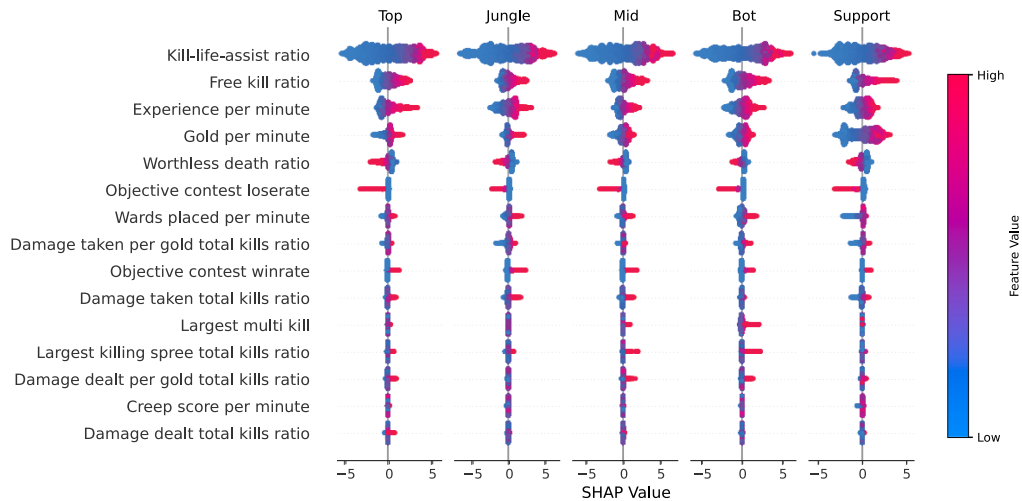


Fig. 4. Relative importance of features for each role-based XGBoost model using SHAP values. The features are ordered in descending order based on their average importance across roles.

TABLE IV
ACCURACY AND ECE OF A LOGISTIC REGRESSION MODEL FORECASTING THE GAME OUTCOME FROM THE PLAYERS' SKILL RATINGS

Model	Accuracy (%) $\uparrow$			ECE (%) $\downarrow$		
	All	Intra	Inter	All	Intra	Inter
PScore + EWMA	63.08	63.35	52.34	1.44	1.57	5.70
PScore + OpenSkill	65.56	65.51	67.56	1.53	1.39	7.66
PScore + Meta_OpenSkill	65.17	65.06	69.23	1.01	1.01	5.47
PScore + FFA_OpenSkill	64.88	64.95	61.87	0.82	0.85	4.64
PScore + Meta_FFA_OpenSkill	65.08	64.94	70.57	1.01	1.00	4.85
PScore + Meta_FFA_TrueSkill	65.22	65.05	69.50	2.94	2.83	6.95
PI + EWMA	63.51	63.73	54.52	1.85	1.96	4.59
PI + Meta_FFA_OpenSkill	65.28	65.13	71.40	1.05	1.01	4.93
PlayeRank + EWMA	62.57	62.85	51.34	2.03	2.22	6.57
PlayeRank + Meta_FFA_OpenSkill	65.00	64.86	70.57	1.09	0.99	5.66

Results are described for all the games, or for intraregion or interregion games only.

we shifted our window by one month and repeated the training-testing evaluation.

Table IV shows the performance of the models in terms of accuracy and ECE. Results are detailed for both intraregion and interregion matches. We can observe the following.

- 1) All the OpenSkill (or TrueSkill) variations have a higher accuracy than their EWMA counterparts. This shows the benefits of having Bayesian ratings with updates depending on the skill ratings of the opponents.
- 2) All the models implementing the Meta OpenSkill variation see a large increase in accuracy in inter-region matches, without sacrificing much in intraregion accuracy. Interregion accuracy becomes higher than intraregion accuracy, which makes sense given how spread apart the skill of different regions can be (and thus easier to predict), as shown by Fig. 5.
- 3) The FFA OpenSkill variation observes a small decrease in accuracy compared to the non-FFA OpenSkill model. This is not surprising, as for this experiment, only a good overall team rating is needed.

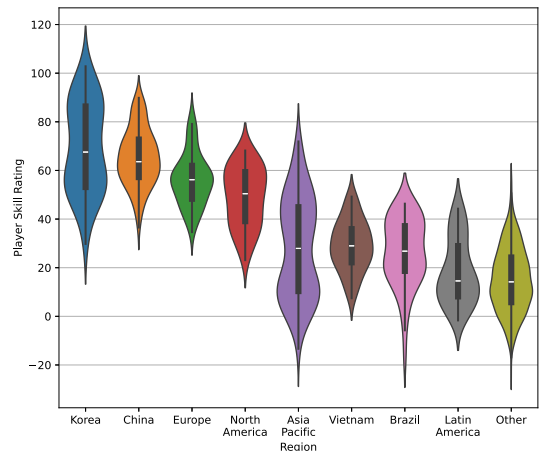


Fig. 5. Distribution of players' skill ratings per region using the Meta_FFA_OpenSkill rating model with PScore.

- 4) The improvements of the proposed rating model transfer well to other game performance models, as shown

TABLE V
RANKING FOR THE FIRST 50 PLAYERS USING THE LOWER BOUND OF THEIR SKILL RATING

Rank	Player	Team	Region	Role	Rating	Rank	Player	Team	Region	Role	Rating
1	Chovy	Gen.G	Korea	Mid	103.76	26	Faker	T1	Korea	Mid	86.69
2	Peyz	Gen.G	Korea	Bot	102.01	27	Missing	JD Gaming	China	Support	85.67
3	Canyon	Gen.G	Korea	Jungle	98.55	28	XUN	BLG ²	China	Jungle	85.39
4	Kiin	Gen.G	Korea	Top	97.02	29	Hans sama	G2 Esports	Europe	Bot	84.79
5	Viper	HLE ¹	Korea	Bot	95.56	30	Creme	Top Esports	China	Mid	84.18
6	Delight	HLE ¹	Korea	Support	93.49	31	JackeyLove	Top Esports	China	Bot	83.96
7	Peanut	HLE ¹	Korea	Jungle	93.42	32	ShowMaker	Dplus KIA	Korea	Mid	83.61
8	Lehends	Gen.G	Korea	Support	92.95	33	Bdd	KT Rolster	Korea	Mid	83.56
9	Zeka	HLE ¹	Korea	Mid	92.83	34	Flandre	JD Gaming	China	Top	82.04
10	Aiming	Dplus KIA	Korea	Bot	91.44	35	BrokenBlade	G2 Esports	Europe	Top	81.95
11	ON	BLG ²	China	Support	91.27	36	Yagao	JD Gaming	China	Mid	81.86
12	Oner	T1	Korea	Jungle	90.68	37	Deft	KT Rolster	Korea	Bot	81.52
13	Elk	BLG ²	China	Bot	90.51	38	Caps	G2 Esports	Europe	Mid	81.15
14	knight	BLG ²	China	Mid	90.07	39	Pyosik	KT Rolster	Korea	Jungle	80.58
15	Keria	T1	Korea	Support	89.86	40	Hang	LNG Esports	China	Support	80.57
16	Gumayusi	T1	Korea	Bot	89.52	41	Scout	LNG Esports	China	Mid	80.56
17	Tian	Top Esports	China	Jungle	89.42	42	GALA	LNG Esports	China	Bot	80.47
18	Lucid	Dplus KIA	Korea	Jungle	88.28	43	BeryL	KT Rolster	Korea	Support	79.85
19	369	Top Esports	China	Top	87.91	44	Yike	G2 Esports	Europe	Jungle	79.72
20	Kanavi	JD Gaming	China	Jungle	87.90	45	Mikyx	G2 Esports	Europe	Support	79.00
21	Doran	HLE ¹	Korea	Top	87.81	46	Kellin	Dplus KIA	Korea	Support	78.95
22	Meiko	Top Esports	China	Support	87.67	47	Tarzan	Weibo Gaming	China	Jungle	78.56
23	Bin	BLG ²	China	Top	87.62	48	Zika	LNG Esports	China	Top	78.51
24	Zeus	T1	Korea	Top	87.35	49	Light	Weibo Gaming	China	Bot	78.42
25	Ruler	JD Gaming	China	Bot	86.89	50	Kingen	Dplus KIA	Korea	Top	77.67

¹ Hanwha Life Esports.

² Bilibili Gaming.

by the PI + Meta_FFA_OpenSkill and PlayeRank + Meta_FFA_OpenSkill model combinations.

5) Compared to OpenSkill, TrueSkill obtains comparable accuracy, but significantly worse calibration.

3) *Skill Ratings Distribution Per Role*: One important aspect of PandaSkill, from the evaluation of players' in-game performance to the computation of their skill ratings, is that all roles are treated identically. To measure this fairness of treatment, we use the Wasserstein distance between the rating distributions of two roles, averaged over every pair of roles. The Wasserstein distance measures the minimal "effort" required to transform one probability distribution into another, effectively quantifying differences between distributions [37].

Results, grouped by PScore model, are as follows.

- 1) *PlayeRank*: Performs poorly, with the best one averaging a Wasserstein distance of 2.45 (SD = 1.32).
- 2) *PI*: Shows good performance, with distances ranging from 0.35 (SD = 0.13) to 0.71 (SD = 0.25).
- 3) *PScore*: Achieves the best performance, with distances between 0.10 (SD = 0.02) and 0.49 (SD = 0.12).

First, both PScore-based and PI-based models significantly outperform PlayeRank-based approaches. This can be attributed to the use of percentile transformations. While PI applies this transformation to the input features, PandaSkill applies it to the output PScores. In both cases, roles are treated more fairly as the scale of either the model inputs or outputs becomes independent of the role. However, from the lower average distance of PScore, we can conclude that doing the percentile transformation at the end, as in PandaSkill, is more effective.

4) *Conclusion*: Bayesian updates via the OpenSkill variants outperform EWMA in forecasting match outcomes. The Meta

component further improves interregion skill rating modeling. While expected because of the nature of the experiment, no positive (nor negative) impact was shown by the addition of the FFA component. Finally, the fairness of treatment built into the PScores translates well into the skill rating estimates, as shown by the low Wasserstein distance across roles.

D. Evaluating the Player Ranking

Table V shows the top 50 players from all the regions. We can see it is dominated by Korea and China, which aligns with most of the interregion results from the past few years (e.g., MSI or Worlds). Only one Western team, G2 Esports, made it into the top 50 ranking.

To perform a quantitative evaluation of the rankings and the underlying skill ratings, as done in the evaluation of PlayeRank [6], we conducted an evaluation by human experts.

1) *Experimental Setting*: We created several surveys, each containing 300 randomly selected pairs of players, and asked experts to choose the better player from each pair. Five surveys were created: four focused on the major regions (Korea, China, Europe, and North America), and one global survey with players from all regions. The experts, recruited from PandaScore's odds traders, were selected based on their expertise in the different LoL regions. Since not all traders were equally familiar with every region, the regional surveys were only completed by those with sufficient knowledge. Table VI details the composition of these surveys.

To analyze the results, we investigated the following two different types of concordance between the model and the experts.

- 1) *Majority concordance*: The fraction of pairs for which a given model agrees with the majority of the experts.

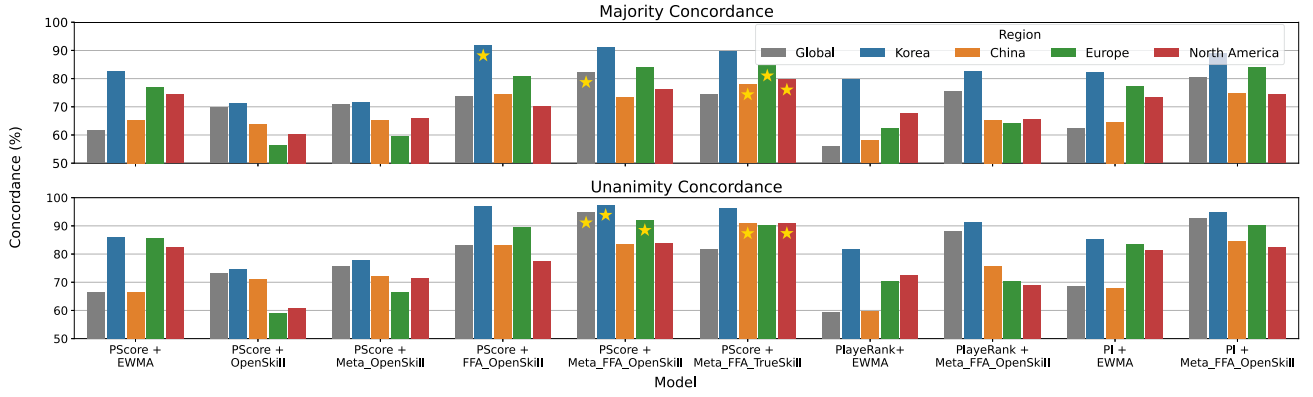


Fig. 6. Majority and unanimity concordance between models and experts per region.

TABLE VI
DESCRIPTION OF PLAYER COMPARISON SURVEYS WITH NUMBER OF EXPERTS,
NUMBER OF UNIQUE PLAYERS, RATIO OF UNANIMOUS ANSWERS, RATIO OF
NO MAJORITY ANSWERS

Region	Experts	Players	Unanimity	No majority
Global	5	477	0.51	-
Korea	4	54	0.70	0.06
China	3	96	0.59	-
Europe	4	52	0.62	0.17
North America	3	40	0.59	-
Total	5	629	0.60	0.05

- 2) *Unanimity concordance*: The fraction of pairs for which a given model agrees with all the experts.

In cases where no majority could be achieved among the experts (e.g., when the number of experts was even, and exactly half of them preferred one player), we chose to drop those comparisons (represented by the *No majority* column in Table VI).

2) *Results*: PScore + Meta_FFA_OpenSkill achieves the highest majority concordance (average of 81.39%, SD = 6.93) and highest unanimity concordance (average of 90.33%, SD = 6.38). PScore + Meta_FFA_TrueSkill is close behind with, on average, a majority concordance of 81.27% (SD = 5.99) and unanimity concordance of 90.05% (SD = 5.31).

Fig. 6 details the results per model and per region as follows.

- 1) EWMA is outperformed by the Meta_FFA_OpenSkill rating model for almost all underlying performance models and regions.
- 2) Using the FFA OpenSkill setting shows a significant improvement in the concordance with the experts, demonstrating the usefulness of incorporating the player's in-game performance in the updates of their skill ratings.
- 3) Using the dual-rating system with a contextual rating and a meta rating, alongside the FFA setting, significantly increases the concordance for the Global survey. This highlights the issue of isolated rating pools coming from the low number of interregion games. PandaSkill provides a working solution to this issue.
- 4) TrueSkill is comparable to OpenSkill, being only notably worse for the Global survey. This indicates that the dual-rating system we propose works better with OpenSkill.

- 5) PI-based player performance ultimately offers very similar skill ratings compared to PScore. This shows that PScore is a valid alternative to PI, with the advantage of being model-agnostic by design.

- 6) Even when improved by the Meta_FFA_OpenSkill rating model, PlayeRank still fails to offer convincing skill ratings compared to other approaches. This underscores the need for a good underlying player performance model.

- 7) Surprisingly, the concordance between experts and the models is very high for the Korea region. This can be explained by the wider range of skill ratings that is within the region compared to the others, as shown by Fig. 5.

3) *Conclusion*: Overall, the expert surveys confirm that the Meta_FFA_OpenSkill approach yields the most trustworthy rankings. The FFA component enhances the standard OpenSkill algorithm by accounting for each player's individual contribution, while the Meta component further refines ratings in a global setting, enabling better cross-region skill estimation. Moreover, Meta_FFA_OpenSkill also proves to be effective at improving both PlayeRank and PI scores over the standard EWMA baseline.

E. Example of the Framework Applied Onto a Game

To demonstrate PandaSkill's practical usage, we propose to look at how it behaves in a given game. We chose game 2 of the match between Hanwha Life Esports (HLE) and T1 in the lower bracket final of the LCK Summer 2024. It was a high-stake game, between two of the strongest teams in the world (see Table V). The game was very close and ended with the victory for T1.

As shown in Table I, despite losing the game, the skill ratings of multiple players from HLE (Viper, Peanut, and Zeka) increased after the game. This was due to the strong performances of these players compared to most of T1's players. Conversely, the skill ratings of some T1 players decreased (Oner, Zeus).

Faker was the most valuable player of his team, having the highest PScore, with a difference of 21.93 compared to Zeka, the player in the same role for HLE (Mid). Looking at the feature contributions to Faker's performance in Fig. 7(f), most of his PScore can be attributed to his exceptional KLA (7.0). Other



Fig. 7. Explanation with SHAP values of PScores of HLE (left) and T1 (right) players in the game 2 of LCK Summer 2024 lower bracket final. (a) Doran (Top) - PScore=32.31. (b) Zeus (Top) - PScore=28.22. (c) Peanut (Jungle) - PScore=69.86. (d) Oner (Jungle) - PScore=43.16. (e) Zeka (Mid) - PScore=59.10. (f) Faker (Mid) - PScore=81.05. (g) Viper (Bot) - PScore=82.55. (h) Gumayusi (Bot) - PScore=73.96. (i) Delight (Support) - PScore=52.94. (j) Keria (Support) - PScore=58.62

outstanding features are his Gold per minute and Largest multi kill.

Looking at the remainder of Fig. 7, of the 15 features, 10 appear among the top four contributors for at least one player. As expected from Fig. 4, KLA has a strong impact for several players, but Free kill ratio, Experience per minute and Gold per minute also frequently rank highly. Globally minor features can still matter locally, as shown by the Largest multi kill for Faker and Damage dealt per gold total kills ratio for Viper. Sometimes a few selected features drive most of the PScore computation (e.g., Faker with KLA), whereas at other times the importance is spread across many (e.g., Keria with the *11 other features*). Overall, these patterns show the flexibility and diversity of PScore computation.

We conducted a small pilot study to test PScore's validity. Five experts (see Table VI) watched the entire game VOD and, for each player, assigned a 0–100 PScore with a small comment. They were asked to consider every aspect they deemed important (e.g., laning, objective control, team-fights, and clutch moments).

For this game, PandaSkill's PScores correlated strongly with the experts' mean scores (Pearson $r = 0.75$, Spearman $\rho = 0.84$), suggesting close alignment between the models and human judgment. Here, are few observations we made from experts' comments and Fig. 7 as follows.

- 1) Faker and Gumayusi received the highest expert scores (95.8 and 87.6), mostly thanks to their exceptional KLA (0 deaths), matching the factor PandaSkill highlighted.
- 2) Zeka and Delight were praised for securing early kills without being punished. Likewise, PandaSkill assigned high SHAP values to the Free kill ratio feature for both players.
- 3) Despite too many deaths, Zeus earned a respectable expert score (68 versus a PScore of 28.22) because his deaths created space for his teammates. This nuance was not captured by the current model. That being said, we should note that Zeus's score was not unanimous, as indicated by the high standard deviation (19.24).

VI. DISCUSSION

Through our experiments, we show that PandaSkill, which combines PScore performance estimates with Meta_FFA_OpenSkill rating updates, produces ratings that are accurate, fair, and interpretable.

First, the role-based models with the percentile transformation capture role-specific nuances without sacrificing comparability, because every role's PScore is expressed on the same scale, a PScore of 75 for a Support is interpreted the same way as a 75 for a Top.

Second, the FFA setting makes skill-rating updates depend on individual contributions rather than on the team's final result. An outstanding performance in a loss is still rewarded with an increase in skill rating, whereas a mediocre performance in a win yields little benefit.

Third, the Meta component addresses the problem of isolated rating pools. It leverages interregion competitions to adjust the

skill of all players within those regions, greatly improving cross-region comparability.

However, we should note a series of limitations in our study that suggest directions for future work.

- 1) *End-game bias*: Because PScore is largely based on player end-game statistics, it cannot capture the full narrative of the game. Event-based metrics (e.g., Free kills ratio) have already proven useful; this idea could be extended to other actions or decisions, such as item choice and timing [27], map movements, or even combat patterns [38]. Another worthy consideration is computing player statistics at specific timestamps, like the 15-min mark, which represents the end of the laning stage [39].
- 2) *Rigid role labels*: While there is a general conception of what a given role brings to the team (e.g., Top provides tankiness), there are no hard rules defining them. A Top player may play a Champion usually played as a Bot if it fits a particular team strategy. When such off-meta strategies are too rare, the available data may be insufficient to model the player's performance accurately. A finer-grained notion of roles, such as play-styles, could help. Play-styles could be derived from Champion archetypes (e.g., "this Champion is a tank") [32], or inferred directly from player actions using supervised [40] or unsupervised methods [6], [19].

VII. CONCLUSION

In this article, we presented PandaSkill, a framework to compute the performance of players in a game and their skill ratings. Compared to other approaches in the literature, PandaSkill fairly compares players from different roles, is model-agnostic, and addresses the isolated rating pools issue common in esports. We demonstrated PandaSkill's usefulness on five years of professional LoL.

We have open-sourced PandaSkill so that it can be reused by the community, driving future research. Here, are some directions we think could be worth studying, because our approach is model-agnostic, more complex models, such as neural networks could be envisaged [10]. The input features to the model are as important, if not more so, than the model itself. The usefulness of more complex features linked to the impact of player actions could be worth exploring [11], [15], [27]. Last but not least, applying PandaSkill to other esports titles, such as *Counter-Strike* or *Dota 2* should yield interesting results. While similar to LoL in essence, both games have specific aspects that would need to be addressed. For instance, the conception of roles is quite different, potentially being more fluid and less defined than in LoL [9], [19]. Moreover, the competitive scenes are fragmented differently. Instead of being fragmented by region, they are fragmented by tiers of competition.

REFERENCES

- [1] L. M. Hvattum, "A comprehensive review of plus-minus ratings for evaluating individual players in team sports," *Int. J. Comput. Sci. Sport*, vol. 18, no. 1, pp. 1–23, 2019.

- [2] J. Lasek, Z. Szlávik, and S. Bhulai, "The predictive power of ranking systems in association Football," *Int. J. Appl. Pattern Recognit.*, vol. 1, no. 1, pp. 27–46, 2013.
- [3] L. Bornn, D. Cervone, and J. Fernandez, "Soccer analytics: Unravelling the complexity of "the beautiful game"," *Significance*, vol. 15, no. 3, pp. 26–29, 2018.
- [4] J. Thorn, P. Palmer, and D. Reuther, *The Hidden Game of Baseball: A Revolutionary Approach to Baseball and Its Statistics*. Chicago, IL, USA: University of Chicago Press, 2015.
- [5] J. Brooks, M. Kerr, and J. Gutttag, "Developing a data-driven player ranking in Soccer using predictive model weights," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2016, pp. 49–55.
- [6] L. Pappalardo, P. Cintia, P. Ferragina, E. Massucco, D. Pedreschi, and F. Giannotti, "Playerank: Data-driven performance evaluation and player ranking in Soccer via a machine learning approach," *ACM Trans. Intell. Syst. Technol.*, vol. 10, no. 5, pp. 1–27, 2019.
- [7] R. Rein and D. Memmert, "Big data and tactical analysis in elite Soccer: Future challenges and opportunities for sports science," *SpringerPlus*, vol. 5, pp. 1–13, 2016.
- [8] T. Decroos, L. Bransen, J. Van Haaren, and J. Davis, "Actions speak louder than goals: Valuing player actions in Soccer," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. data mining*, 2019, pp. 1851–1861.
- [9] P. Varga, T. M. Scholz, and E. T. Tan, "Esports player analytics," in *Routledge Handbook Esports*, Routledge, 2024, pp. 191–202.
- [10] O. Delalleau, E. Contal, E. Thibodeau-Laufer, R. C. Ferrari, Y. Bengio, and F. Zhang, "Beyond skill rating: Advanced matchmaking in Ghost Recon online," *IEEE Trans. Comput. Intell. AI Games*, vol. 4, no. 3, pp. 167–177, Sep. 2012.
- [11] P. Z. Maymin, "Smart kills and worthless deaths: Esports analytics for League of Legends," *J. Quantitative Anal. Sports*, vol. 17, no. 1, pp. 11–27, 2021.
- [12] E. Kleinman and M. S. El-Nasr, "Using data to "git gud": A push for a player-centric approach to the use of data in esports," in *Proc. EHPHCl: Esports High- Perform. HCI Workshop, 2021 CHI Conf. Hum. Factors Comput. Syst.*, OSF Preprints, 2021.
- [13] B. Sabtan, S. Cao, and N. Paul, "Current practice and challenges in coaching esports players: An interview study with League of Legends professional team coaches," *Entertainment Comput.*, vol. 42, 2022, Art. no. 100481.
- [14] B. T. Sharpe, N. Besombes, M. R. Welsh, and P. D. Birch, "Indexing esports performance," *J. Electron. Gaming Esports*, vol. 1, no. 1, 2022.
- [15] P. Xenopoulos, H. Doraiswamy, and C. Silva, "Valuing player actions in Counter-Strike: Global offensive," in *Proc. IEEE Int. Conf. Big Data*, 2020, pp. 1283–1292.
- [16] S. Pradhan and Y. Abdourzakou, "Power ranking" professional circuit esports teams using multi-criteria decision-making (MCDM)," *J. Sports Anal.*, vol. 6, no. 1, pp. 61–73, 2020.
- [17] J. Gudmundsson and M. Horton, "Spatio-temporal analysis of team sports," *ACM Comput. Surv.*, vol. 50, no. 2, pp. 1–34, 2017.
- [18] L. Pappalardo et al., "A public data set of spatio-temporal match events in Soccer competitions," *Sci. Data*, vol. 6, no. 1, 2019, Art. no. 236.
- [19] S. Demediuk et al., "Performance index: A new way to compare players," in *Proc. 2021 MIT Sloan Sports Anal. Conf.*, 2021.
- [20] A. E. Elo, *The Rating of Chessplayers, Past and Present*. New York, NY, USA: Arco Pub., 1978. [Online]. Available: <http://www.amazon.com/Rating-Chess-Players-Past-Present/dp/0668047216>
- [21] R. Herbrich, T. Minka, and T. Graepel, "Trueskill: A Bayesian skill rating system," *Adv. Neural Inf. Process. Syst.*, vol. 19, 2006.
- [22] V. Joshy, "Openskill: A faster asymmetric multi-team, multiplayer rating system," *J. Open Source Softw.*, vol. 9, no. 93, 2024, Art. no. 5901, doi: [10.21105/joss.05901](https://doi.org/10.21105/joss.05901).
- [23] M. E. Glickman and A. C. Jones, "Rating the Chess rating system," *Chance-Berlin Then New York-*, vol. 12, pp. 21–28, 1999.
- [24] A. Sapienza, Y. Zeng, A. Bessi, K. Lerman, and E. Ferrara, "Individual performance in team-based online games," *Roy. Soc. open Sci.*, vol. 5, no. 6, 2018, Art. no. 180329.
- [25] Introducing rating 2.1. Accessed: Oct. 23, 2024. [Online]. Available: <https://www.hltv.org/news/40051/introducing-rating-21>
- [26] H. Xu and S. Moka, "Rating players of Counter-Strike: Global offensive based on plus/minus value," 2024, *arXiv:2409.05052*.
- [27] J. Jang, J. Y. Woo, and H. K. Kim, "Action2score: An embedding approach to score player action," *Proc. ACM Hum. - Comput. Interact.*, vol. 6, no. CHI PLAY, pp. 1–23, 2022.
- [28] M. E. Glickman, "The glicko system," *Boston Univ.*, vol. 16, no. 8, p. 9, 1995.
- [29] M. E. Glickman, "Example of the glicko-2 system," *Boston Univ.*, vol. 28, 2012.
- [30] T. Minka, R. Cleven, and Y. Zaykov, "Trueskill 2: An improved Bayesian skill rating system," Tech. Rep., 2018.
- [31] R. C. Weng and C.-J. Lin, "A Bayesian approximation method for online ranking," *J. Mach. Learn. Res.*, vol. 12, no. 1, 2011.
- [32] M. Suznjevic, M. Matijasevic, and J. Konfic, "Application context based algorithm for player skill evaluation in Moba games," in *Proc. 2015 Int. Workshop Netw. Syst. Support Games*, 2015, pp. 1–6.
- [33] Fandom, "Leaguepedia API documentation," 2024, Accessed: Sep. 15, 2024. [Online]. Available: https://lol.fandom.com/wiki/Help:Leaguepedia_API
- [34] T. S. Filho, H. Song, M. Perello-Nieto, R. Santos-Rodriguez, M. Kull, and P. Flach, "Classifier calibration: A survey on how to assess and improve predicted class probabilities," *Mach. Learn.*, vol. 112, no. 9, pp. 3211–3260, 2023.
- [35] S. Lundberg, "A unified approach to interpreting model predictions," 2017, *arXiv:1705.07874*.
- [36] H. Lee, "An implementation of the trueskill rating system for Python," GitHub repository, 2018. [Online]. Available: <https://github.com/sublee/trueskill>
- [37] V. M. Panaretos and Y. Zemel, "Statistical aspects of Wasserstein distances," *Annu. Rev. Statist. Appl.*, vol. 6, no. 1, pp. 405–431, 2019.
- [38] P. Yang, B. E. Harrison, and D. L. Roberts, "Identifying patterns in combat that are predictive of success in Moba games," in *Proc. FDG*, 2014.
- [39] K. Arik, "Machine learning models on Moba gaming: League of Legends winner prediction," *Acta Infologica*, vol. 7, no. 1, pp. 139–151, 2023.
- [40] C. Eggert, M. Herrlich, J. Smeddinck, and R. Malaka, "Classification of player roles in the team-based multi-player game Dota 2," in *Proc. Entertainment Comput.-ICEC 2015: 14th Int. Conf.*, ICEC 2015, Trondheim, Norway, 2015, Sep. 2015, pp. 112–125.